

Technical Documentation

Smart Home Control System Using Composite Design Pattern

1. System Overview

The Smart Home Control System models a hierarchical home environment that contains floors, rooms, and individual smart devices. The system enables the same operations to be applied to a single device or to a complete group of devices, such as a room, a floor, or the entire home.

The design is based on the Composite Design Pattern because the smart home naturally follows a part-whole tree structure. A smart home contains floors, floors contain rooms, and rooms contain devices.

2. Hierarchical Structure

The implementation builds the following hierarchy:

```
Smart Home
├── First Floor
│   ├── Living Room
│   │   ├── Living Room Light
│   │   ├── Smart TV
│   │   └── Air Conditioner
│   └── Kitchen
│       ├── Kitchen Light
│       └── Smart Oven
└── Second Floor
    ├── Bedroom
    │   ├── Bedroom Light
    │   └── Heater
    └── Security Area
        ├── Security Camera
        └── Smart Door Lock
```

3. How the Composite Pattern Is Applied

The Composite Design Pattern allows individual objects and collections of objects to be treated uniformly. In this project, both a single smart device and a smart group support the same common operations through the abstract class SmartComponent.

Operation	Purpose
turnOn()	Turns on one device, or recursively turns on every component inside a group.
turnOff()	Turns off one device, or recursively turns off every component inside a group.
showStatus()	Displays the status of one device, or recursively displays the status of every component in a group.
getPowerConsumption()	Returns the active power usage of one device, or the total active power usage of a group.

4. Pattern Roles in the Solution

Composite Pattern Role	Project Class	Responsibility
Component	SmartComponent	Defines the shared interface for smart devices and smart groups.
Leaf	SmartDevice	Represents an individual smart device that cannot contain child components.
Composite	SmartGroup	Stores child components and applies operations recursively to them.
Client	Main	Creates the hierarchy and demonstrates the pattern behavior.

5. Class Responsibilities

5.1 SmartComponent - Component

SmartComponent is an abstract class that declares the common operations shared by devices and groups. It also defines add() and remove() methods. These methods are overridden in SmartGroup, while the default implementation prevents unsupported child management in non-composite objects.

5.2 SmartDevice - Leaf

SmartDevice represents a single device such as a light, TV, heater, oven, camera, or smart lock. It stores its name, active power consumption, and current ON/OFF status. It implements the shared operations directly.

5.3 SmartGroup - Composite

SmartGroup represents a container such as the smart home, a floor, a room, or a security area. It stores a List<SmartComponent> and recursively calls operations on all child components. This enables nested groups and uniform behavior across the hierarchy.

5.4 Main - Client

Main creates all smart groups and smart devices, attaches them using `add()`, and demonstrates key operations: displaying status before activation, turning on the entire home, calculating power consumption, turning off only the second floor, and recalculating total power usage.

6. UML Relationship Explanation

Relationship	Explanation
Inheritance / Generalization	SmartDevice and SmartGroup inherit from SmartComponent.
Aggregation / Containment	SmartGroup contains multiple SmartComponent objects using <code>List<SmartComponent></code> .
Client Dependency	Main uses SmartComponent, SmartGroup, and SmartDevice to build and control the smart-home structure.

7. Why the Pattern Fits This System

- The system has a clear tree-like structure: home -> floors -> rooms -> devices.
- The same operation can be applied to a single device or to an entire group.
- The recursive design keeps the code readable and avoids duplicated control logic.
- New rooms, floors, or devices can be added without changing the overall structure.
- The design improves maintainability, scalability, and clarity.

8. Conclusion

The UML diagram and implementation provide a clear application of the Composite Design Pattern. The solution treats individual smart devices and groups of devices uniformly while supporting recursive control and power calculation across a hierarchical smart-home structure. This makes the design organized, extendable, and suitable for the project requirements.